CloudNotes

Full DevOps Showcase Application

Course: INT377 Cloud Computing and DevOps Essentials

Institution: Lovely Professional University

Submitted By: Savinay Singh

Date: May 26, 2026

Submission Item	Value
Project Type	Containerized cloud-native note-taking application with DevOps lifecycle automation
Primary Objective	Demonstrate Docker, Kubernetes, Terraform, Jenkins, Prometheus, Grafana, AWS, testing, and security practices
Repository Root	C:\Users\savin\Documents\08_Archives
Current Branch / Commit	main / bfcf6b3 Auto-update App IP after Terraform Apply: 3.110.169.107
Submission Status	PDF-ready professional documentation for course evaluation

This report was generated after inspecting the project source, infrastructure files, automation scripts, monitoring configuration, tests, and available runtime evidence. Secrets and private keys are intentionally not reproduced.

Table of Contents

3. Executive Summary 5

4. Course Mapping and Learning Outcomes 6

Learning outcome assessment . . . 6

5. Project Overview and Objectives 7

5.1 Project description . . . 7

5.2 Core features . . . 7

5.3 Project goals . . . 7

6. System Architecture 8

6.1 High-level delivery flow . . . 8

6.2 Deployment models supported . . . 8

6.3 Design observations . . . 8

7. Technology Stack and Components 9

7.1 Backend application . . . 9

7.2 REST API endpoints . . . 9

7.3 Frontend application . . . 9

7.4 Container and infrastructure stack . . . 10

8. Application Features and Functionality 11

8.1 Database schema . . . 11

8.2 Functional workflow . . . 11

9. Deployment Guide 13

9.1 Local development with Docker Compose . . . 13

9.2 Kubernetes deployment with Minikube . . . 13

9.3 AWS deployment . . . 13

10. BAT Deployment Automation 14

10.1 start_mumbai.bat . . . 14

10.2 start_automated_terraform.ps1 . . . 14
10.3 stop_mumbai.bat . . . 14
10.4 Why the BAT files matter . . . 14
11. Infrastructure as Code **16**
11.1 Cloud economics and cleanup . . . 16
11.2 IaC accuracy notes . . . 16
12. Docker and Kubernetes Orchestration **17**
12.1 Docker Compose services . . . 17
12.2 Kubernetes implementation . . . 17
13. Continuous Integration and Deployment **19**
13.1 Jenkins configuration . . . 19
14. Monitoring, Logging and Observability **21**
14.1 Prometheus . . . 21
14.2 Alert rules . . . 21
14.3 Grafana dashboard . . . 22
15. Security and Best Practices **23**
15.1 Security considerations observed . . . 23
16. Testing and Verification **24**
16.1 Backend unit tests . . . 24
16.2 End-to-end browser automation . . . 24
16.3 Infrastructure verification . . . 24
17. Challenges, Solutions and Lessons Learned **25**
17.1 Future improvements . . . 25
18. Course Outcomes Assessment **26**
19. Visual Documentation **27**
20. Conclusion and Recommendations **29**
20.1 Final recommendations . . . 29

Appendix A. Key File Inventory **30**

Appendix B. Selected source excerpts 30

3. Executive Summary

CloudNotes is a full-stack DevOps demonstration project built around a practical note-taking application. The application itself is intentionally simple: users can create, read, update, delete, and search notes. The value of the project is that this small product surface is surrounded by a complete cloud and DevOps workflow: containerized services, Docker Compose local runtime, Kubernetes manifests, Terraform infrastructure, Jenkins CI/CD, Prometheus metrics, Grafana dashboards, tests, and one-click AWS deployment scripts.

The project demonstrates the DevOps lifecycle from source code to production-like infrastructure. A code change can move from GitHub into Jenkins, through backend tests, secret scanning, Docker image creation, image vulnerability scanning, Docker Hub publishing, and SSH-based deployment onto an AWS EC2 application host. The deployed stack is observable through Prometheus and Grafana, and the cloud environment is created and destroyed through Terraform.

The current repository covers INT377 Units I through V comprehensively and Unit VI partially. Unit I is represented by GitHub source control and cloud fundamentals; Unit II by Docker and Kubernetes; Unit III by Terraform and AWS; Unit IV by Jenkins pipeline automation; Unit V by Prometheus, Grafana, security scanning, and alert rules; Unit VI by autoscaling, performance monitoring, compliance-oriented cleanup, and cloud-cost awareness.

Key achievement	Evidence from project
Application built and containerized	Flask backend, Nginx frontend, PostgreSQL, Dockerfiles, docker/docker-compose.yml
Infrastructure automated	terraform/main.tf, modules/vpc, modules/ec2, modules/s3, billing_alarm.tf
CI/CD implemented	Jenkinsfile with checkout, test, scan, build, push, deploy, and health-check stages
Monitoring implemented	Prometheus scrape config, alert rules, and Grafana dashboard provisioning
Deployment lifecycle automated	start_mumbai.bat, start_automated_terraform.ps1, stop_mumbai.bat
Testing verified	9 backend pytest tests passed; Puppeteer E2E script exists for browser evidence capture

4. Course Mapping and Learning Outcomes

The following matrix maps the INT377 syllabus units to actual CloudNotes files and behavior. The evidence column references concrete project files rather than generic DevOps claims.

Course Unit	Topic	Implementation in CloudNotes	Evidence
Unit I	Cloud fundamentals, Git, DevOps introduction	Repository-based workflow, GitHub remote, live AWS deployment, environment-oriented documentation	README.md, HOW_TO_RUN.md, Jenkinsfile
Unit II	Virtualization, Docker, Kubernetes	Dockerized frontend/backend/database/monitoring services and Kubernetes deployments, services, HPA, Ingress, PVC, Secret	app/*/Dockerfile, docker/docker-compose.yml, kubernetes/*.yaml
Unit III	Infrastructure as Code and AWS	Terraform provisions VPC, public subnet, Internet Gateway, route table, security groups, EC2, S3, SNS, and billing alarm	terraform/main.tf, provider.tf, variables.tf, modules/*, billing_alarm.tf
Unit IV	CI/CD pipelines	Jenkins pipeline runs tests, scans, builds Docker images, pushes to Docker Hub, and deploys to EC2	Jenkinsfile, jenkins/jenkins.yaml, jenkins/Dockerfile.local
Unit V	Monitoring, security, DevSecOps	Prometheus metrics and alerts, Grafana dashboard, TruffleHog secret scan, Trivy image scan, Jenkins credentials injection	monitoring/*, Jenkinsfile, jenkins/init.groovy.d/creds.groovy
Unit VI	Advanced topics and performance management	Horizontal Pod Autoscaler, billing alarm, P95 latency panel, cloud cleanup workflow	kubernetes/hpa.yaml, terraform/billing_alarm.tf, cloudnotes.json, stop_mumbai.bat

Learning outcome assessment

- CO1: Explains cloud deployment through AWS EC2, VPC, public subnet, S3, SNS, and CloudWatch billing alarm resources.
- CO2: Demonstrates containerization through separate backend and frontend images, a multi-service Docker Compose stack, and Kubernetes orchestration manifests.
- CO3: Applies Infrastructure as Code by defining cloud resources as version-controlled Terraform modules.
- CO4: Implements CI/CD as code through Jenkins stages that transform source changes into deployed containers.
- CO5: Applies observability and DevSecOps through Prometheus, Grafana, alert rules, TruffleHog, Trivy, and credential injection.
- CO6: Introduces advanced operational concerns through HPA scaling, cost control, performance panels, and teardown automation.

5. Project Overview and Objectives

5.1 Project description

CloudNotes is a containerized web application for managing notes. The user interface is implemented with HTML, CSS, and vanilla JavaScript, served by Nginx. The backend is a Flask API that persists notes through SQLAlchemy to PostgreSQL in containerized environments. The project is not only an application; it is a complete DevOps showcase designed to demonstrate repeatable infrastructure, automated delivery, and operational monitoring.

5.2 Core features

- Create notes with a title and body.
- Read all notes with timestamps.
- Update existing notes through a modal edit interface.
- Delete notes and refresh the list.
- Search notes by title or content through the backend search query.
- Expose backend health at /api/health and Prometheus metrics at /metrics.
- Run locally, in Kubernetes, or on AWS through Terraform and Docker Compose.

5.3 Project goals

Goal	How CloudNotes addresses it
Containerization	Separates frontend, backend, database, Prometheus, and Grafana into services.
IaC	Uses Terraform modules so infrastructure can be recreated without manual console work.
Automation	Provides BAT/PowerShell launch and cleanup scripts around Terraform and Jenkins configuration.
CI/CD	Uses Jenkins pipeline as code with test, scan, build, push, and deploy phases.
Observability	Collects Flask and custom business metrics through Prometheus and visualizes them in Grafana.
Security awareness	Includes CI secret scanning, image scanning, Kubernetes Secrets, and clear production-hardening recommendations.

6. System Architecture

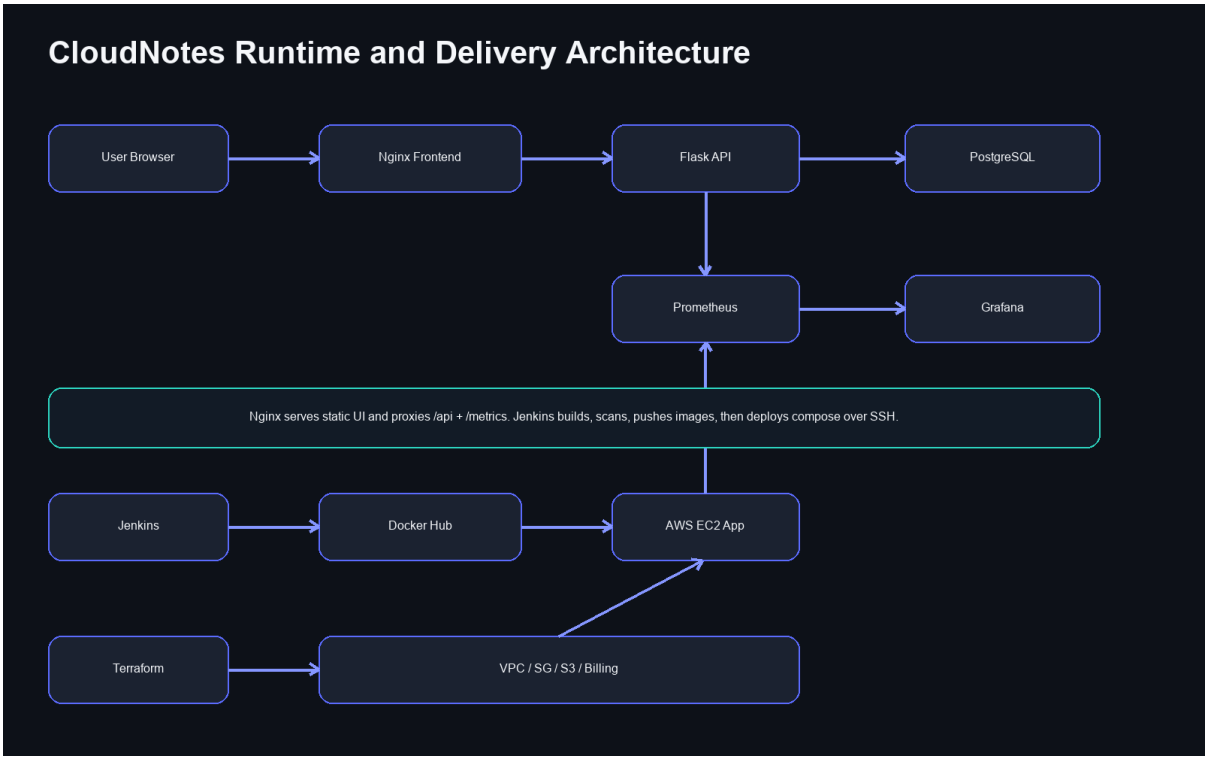


Figure 1. CloudNotes runtime and delivery architecture.

The architecture follows a layered model. User traffic reaches the Nginx frontend, which serves static assets and proxies API requests to the Flask backend. The backend stores data in PostgreSQL and publishes metrics. Prometheus scrapes the backend metrics endpoint and Grafana reads from Prometheus to present dashboards. Jenkins performs build and deployment automation. Terraform provisions the AWS environment that hosts the Jenkins and application EC2 instances.

6.1 High-level delivery flow

```
Developer code push -> GitHub -> Jenkins webhook -> Checkout -> Backend tests -> Security scan
-> Docker build -> Trivy image scan -> Docker Hub push -> SSH deploy to AWS app server ->
Prometheus scrape -> Grafana visualization
```

6.2 Deployment models supported

Model	Runtime	Purpose	Key command or file
Local Docker Compose	PostgreSQL, backend, frontend, Prometheus, Grafana on one Docker bridge network	Fast local development and demonstration	docker-compose -f docker/docker-compose.yml up --build -d
Kubernetes / Minikube	Namespace, deployments, services, ingress, PVC, secrets, HPA	Container orchestration and self-healing demonstration	kubectl apply -f kubernetes/
AWS production-like	Terraform-created EC2 instances running Docker Compose plus Jenkins host	Cloud deployment and CI/CD demonstration	start_mumbai.bat

6.3 Design observations

- The live AWS path uses one public subnet in ap-south-1a, not a multi-AZ subnet pair. This is suitable for coursework but should be expanded for production high availability.
- The app server hosts the application and monitoring stack together. This keeps the demo compact but couples operational tooling to application availability.
- Jenkins runs on a separate EC2 instance, which cleanly separates delivery automation from the app runtime.
- The repository also includes a Kubernetes path to demonstrate orchestration independently from the AWS Compose deployment.

7. Technology Stack and Components

7.1 Backend application

Backend item	Actual implementation
Framework	Flask 3.0.0
ORM	Flask-SQLAlchemy 3.1.1 / SQLAlchemy model Note
Database drivers	psycopg2-binary 2.9.9 for PostgreSQL; SQLite fallback for local/default mode
Runtime server	Gunicorn 21.2.0 with 2 workers in the backend Dockerfile
Metrics	prometheus-flask-exporter and prometheus_client custom counters
Tests	pytest 7.4.3; 9 backend tests currently pass

```
from flask import Flask, jsonify, request, Response
from flask_sqlalchemy import SQLAlchemy
from flask_cors import CORS
from prometheus_flask_exporter import PrometheusMetrics
from prometheus_client import Counter, generate_latest, CONTENT_TYPE_LATEST
from datetime import datetime, timezone
import os

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = os.getenv(
    'DATABASE_URL', 'sqlite:///notes.db'
)
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)
CORS(app)  # Enable CORS for all routes
metrics = PrometheusMetrics(app, path=None) # Disable automatic path

# Custom metrics
notes_created_counter = Counter(
    'notes_created_total',
    'Total number of notes created'
)
... excerpt truncated for report clarity ...
```

7.2 REST API endpoints

Endpoint	Method	Purpose
/api/health	GET	Health check used by Kubernetes probes and CI deployment checks
/api/notes	GET	List all notes or search notes with the search query parameter
/api/notes	POST	Create a new note; increments notes_created_total
/api/notes/<id>	PUT	Update an existing note; increments notes_updated_total
/api/notes/<id>	DELETE	Delete a note; increments notes_deleted_total
/metrics	GET	Expose Prometheus metrics in text exposition format

7.3 Frontend application

The frontend uses HTML5, CSS3, and vanilla JavaScript. The UI includes a centered app shell, search input, create form, note list, edit modal, loading spinner, and toast notifications. app/frontend/style.css implements a dark glassmorphism visual design with responsive behavior. app/frontend/app.js implements debounced search, CRUD fetch calls, HTML escaping for rendered note content, and keyboard support for Ctrl+Enter plus Escape.

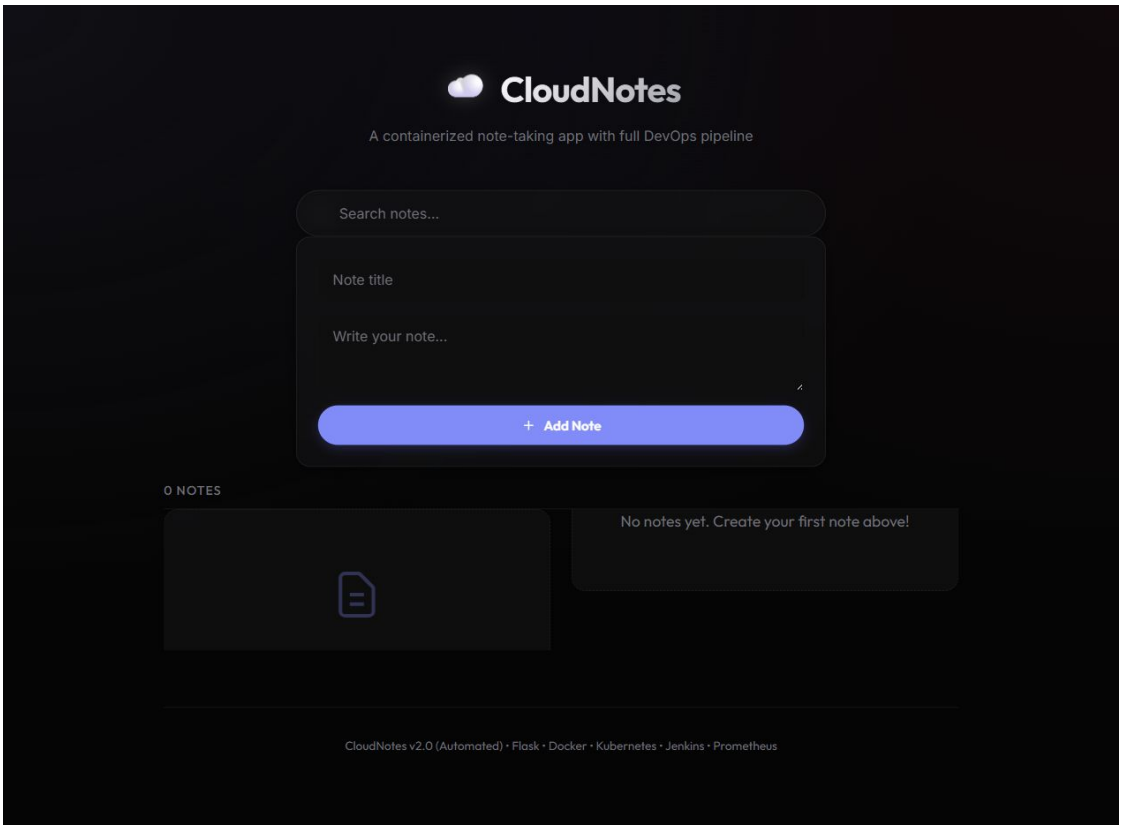


Figure 2. CloudNotes application initial UI.

7.4 Container and infrastructure stack

Layer	Technology	Role
Frontend container	Nginx Alpine	Serves static assets and proxies /api plus /metrics
Backend container	Python 3.11 slim	Runs Gunicorn and Flask API
Database	PostgreSQL 15 Alpine	Stores note records
Local orchestration	Docker Compose	Starts app and monitoring stack together
Cluster orchestration	Kubernetes	Deployments, Services, HPA, PVC, Ingress
Cloud infrastructure	Terraform + AWS	VPC, subnet, EC2, S3, SNS, CloudWatch billing alarm
CI/CD	Jenkins	Build, test, scan, push, deploy
Observability	Prometheus + Grafana	Metrics collection and visualization

8. Application Features and Functionality

CloudNotes demonstrates a complete CRUD workflow with a small domain model. This is useful for DevOps coursework because each user action can be traced through the browser, Nginx proxy, Flask route, SQLAlchemy model, PostgreSQL persistence layer, metrics counters, Prometheus scrape, and Grafana panel.

8.1 Database schema

```
Note table
- id: Integer primary key
- title: String(100), required
- body: Text, required
- created_at: DateTime, default UTC timestamp
```

8.2 Functional workflow

Feature	User interaction	Backend behavior	Metric impact
Create	Enter title/body and click Add Note	POST /api/notes validates required fields and inserts row	notes_created_total increments
Read	Open dashboard or refresh after mutation	GET /api/notes returns JSON list	HTTP request metrics update
Search	Type into search field	GET /api/notes?search=... filters title/body	HTTP request metrics update
Update	Open edit modal and save	PUT /api/notes/<id> updates row	notes_updated_total increments
Delete	Click Delete	DELETE /api/notes/<id> removes row	notes_deleted_total increments

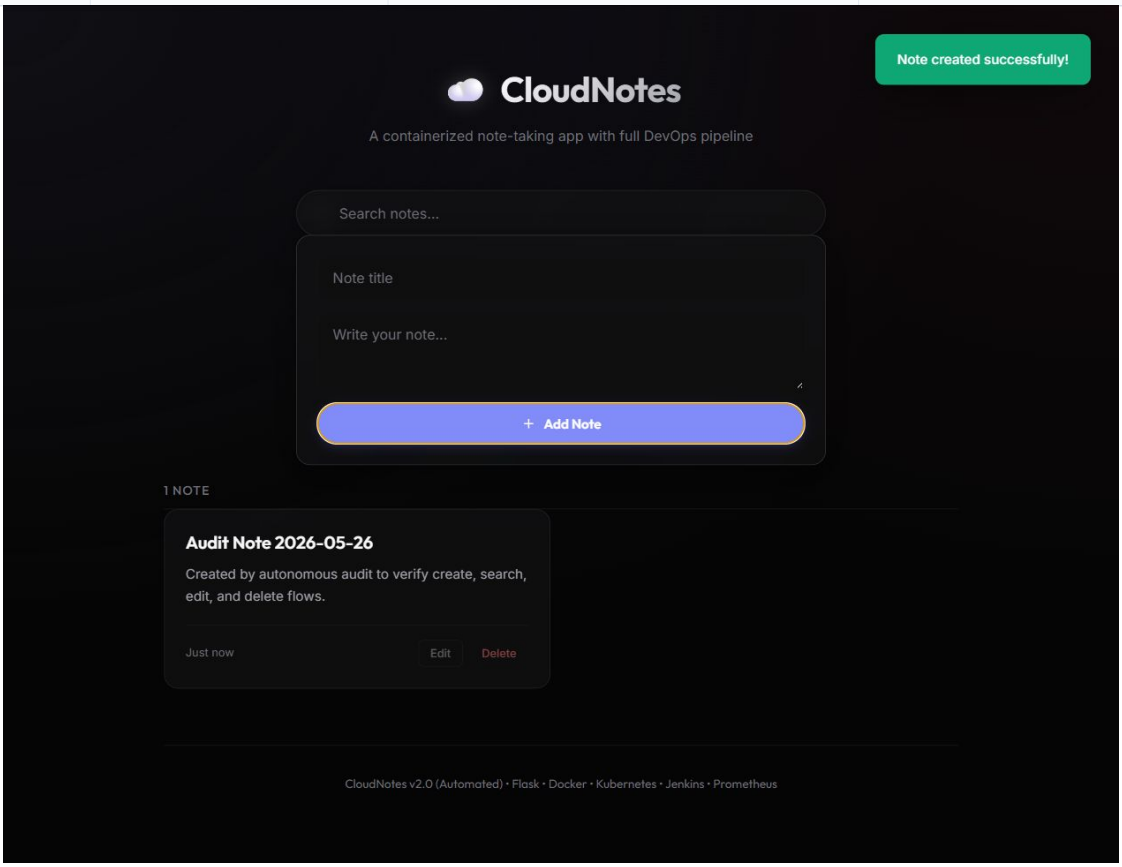


Figure 3. Note creation workflow evidence.

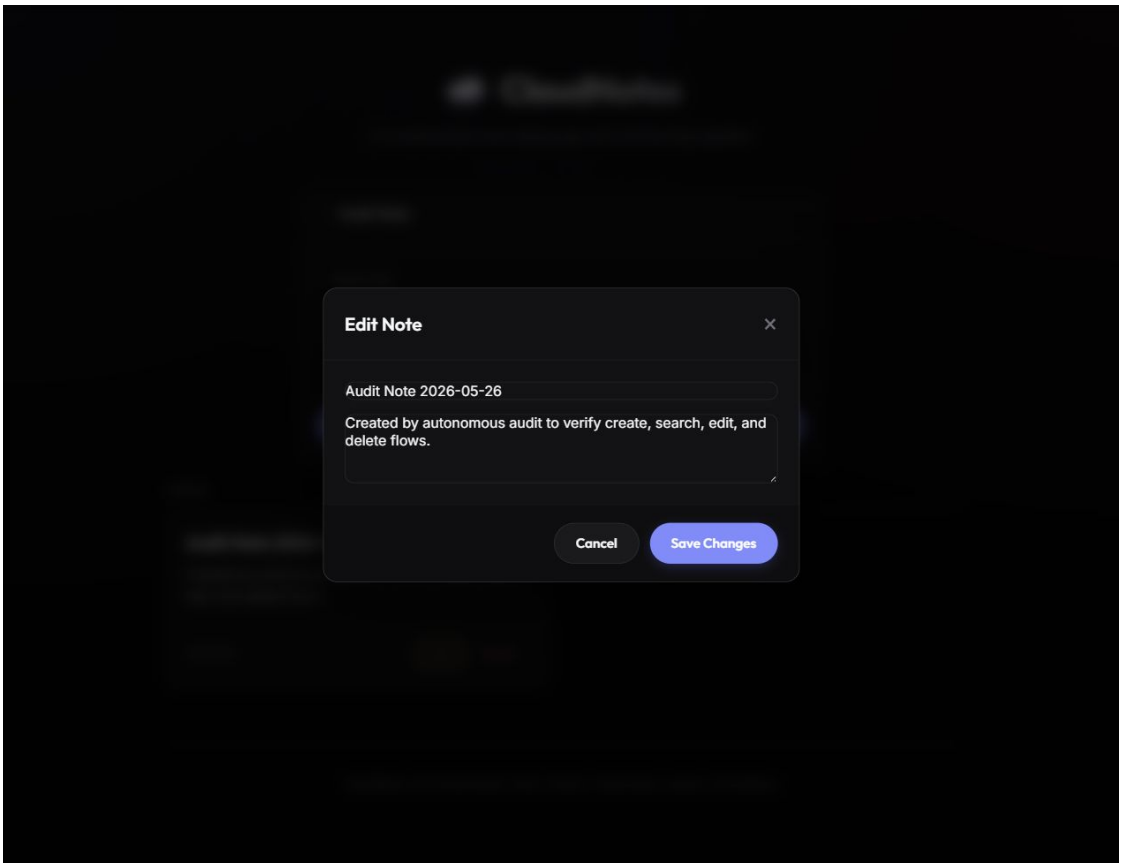


Figure 4. Edit modal workflow evidence.

9. Deployment Guide

9.1 Local development with Docker Compose

```
1. Start Docker Desktop.
2. From the repository root, run:
   docker-compose -f docker/docker-compose.yml up --build -d
3. Open the app at http://localhost:80
4. Open Grafana at http://localhost:3001 with admin/admin
5. Open Prometheus at http://localhost:9091
6. For recovery, run:
   powershell -File docker/recover.ps1
```

9.2 Kubernetes deployment with Minikube

```
1. minikube start --driver=docker
2. kubectl apply -f kubernetes/
3. kubectl get pods -n cloudnotes
4. minikube service frontend -n cloudnotes
5. kubectl logs -f deployment/backend -n cloudnotes
```

9.3 AWS deployment

The AWS deployment path is intentionally automated. The student runs start_mumbai.bat from Windows, which invokes start_automated_terraform.ps1. Terraform creates the infrastructure, EC2 user_data bootstraps Docker/Jenkins/app services, the script installs Jenkins credentials, updates the GitHub webhook, and prints URLs for the app, Jenkins, Grafana, and Prometheus.

Deployment mode	Main access points
Local	App: localhost:80; Backend: localhost:5000; Grafana: localhost:3001; Prometheus: localhost:9091
Kubernetes	Frontend service through Minikube or Ingress host cloudnotes.local
AWS live run evidence	App IP 3.111.149.43 and Jenkins IP 3.110.161.177 appeared in analysis_artifacts/start_mumbai.log during the captured run

10. BAT Deployment Automation

10.1 start_mumbai.bat

```
@echo off
powershell.exe -ExecutionPolicy Bypass -File "%~dp0start_automated_terraform.ps1"
pause
```

start_mumbai.bat is the one-click entry point for Windows users. It bypasses the PowerShell execution policy for the project script and then pauses so the student can read the terminal output. The real automation lives in start_automated_terraform.ps1.

10.2 start_automated_terraform.ps1

Step	Automation behavior
Secret loading	Reads GitHub and Docker Hub secrets from environment variables or .secrets files.
Terraform apply	Runs terraform -chdir=terraform init and apply -auto-approve.
Output extraction	Reads app_public_ip and jenkins_public_ip from Terraform outputs.
Jenkinsfile update	Rewrites APP_SERVER_IP in Jenkinsfile to match the latest app EC2 IP.
Credential installation	Copies Jenkins Groovy init script, Docker Hub password, and app SSH key to Jenkins host and restarts Jenkins.
Webhook patch	Updates the GitHub webhook URL to the new Jenkins public IP.
Git synchronization	Commits and pushes Jenkinsfile if the IP changed.
URL output	Prints App, Jenkins, Grafana, and Prometheus URLs.

10.3 stop_mumbai.bat

```
@echo off
echo ========================================
echo    Destroying Mumbai Infrastructure...
echo ========================================
terraform -chdir=terraform destroy -auto-approve
echo.
echo All resources deleted. Costs: $0.00
pause
```

stop_mumbai.bat is the cost-control counterpart to start_mumbai.bat. It runs terraform destroy -auto-approve inside the terraform directory and then prints that resources were deleted. During the captured run, stop_mumbai.bat destroyed 14 Terraform-managed resources, including EC2 instances, VPC components, S3 bucket resources, SNS topic, billing alarm, and key pair.

10.4 Why the BAT files matter

- They reduce a long manual deployment sequence into repeatable commands.
- They demonstrate Infrastructure as Code in a way that a teacher can run and observe.
- They reduce AWS cost risk because cleanup is part of the project workflow.
- They document how Terraform outputs, Jenkins configuration, and GitHub webhooks connect.
- They show that deployment is not just code packaging; it is environment orchestration.

```
========================================
 Starting Mumbai CloudNotes (Terraform)
========================================
Initializing Terraform...
Applying Terraform Infrastructure (this may take 2-3 mins)...
aws_key_pair.deployer: Refreshing state... [id=cloudnotes-key-new]
module.vpc.aws_vpc.main: Refreshing state... [id=vpc-093102b90f007a382]
module.frontend_bucket.aws_s3_bucket.this: Refreshing state... [id=cloudnotes-frontend-dev]
module.vpc.aws_internet_gateway.gw: Refreshing state... [id=igw-0b80709bf053a369c]
module.vpc.aws_subnet.public: Refreshing state... [id=subnet-0d20555b28963fadf]
module.vpc.aws_security_group.jenkins_sg: Refreshing state... [id=sg-0bc7713514db06340]
module.vpc.aws_security_group.app_sg: Refreshing state... [id=sg-02566b14068ba8def]
module.frontend_bucket.aws_s3_bucket_public_access_block.this: Refreshing state...
[id=cloudnotes-frontend-dev]
aws_sns_topic.billing_alerts: Refreshing state... [id=arn:aws:sns:us-
east-1:168312024417:cloudnotes-billing-alerts]
module.vpc.aws_route_table.public: Refreshing state... [id=rtb-0421e37d2a41bebc4]
module.jenkins_ec2.aws_instance.this: Refreshing state... [id=i-0bd5a6b1fb28cda8e]
module.vpc.aws_route_table_association.public: Refreshing state...
```

```
[id=rtbassoc-0ac110a7e7a8acb04]
module.app_ec2.aws_instance.this: Refreshing state... [id=i-0a2437129a012268d]
aws_cloudwatch_metric_alarm.billing_alarm: Refreshing state... [id=cloudnotes-monthly-billing-
alarm]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration
and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
... excerpt truncated for report clarity ...
```

11. Infrastructure as Code

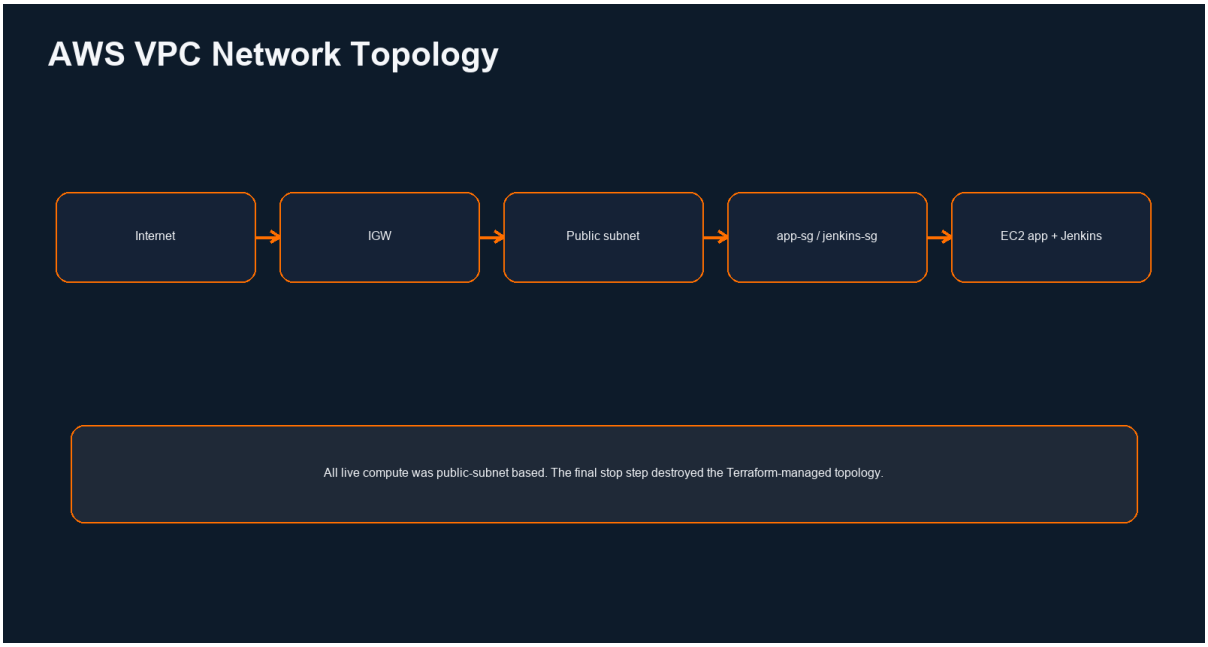


Figure 5. AWS VPC topology generated from project evidence.

Terraform file	Purpose
terraform/provider.tf	Pins AWS provider ~> 5.0, uses ap-south-1 and us-east-1 alias for billing metrics.
terraform/variables.tf	Defines region, instance type t3.small, project name, environment, billing email, billing threshold.
terraform/main.tf	Creates key pair and wires VPC, Jenkins EC2, app EC2, and S3 modules.
terraform/modules/vpc/main.tf	Creates VPC 10.0.0.0/16, public subnet 10.0.1.0/24, IGW, route table, app and Jenkins security groups.
terraform/modules/ec2/main.tf	Defines Ubuntu 22.04 EC2 instances with user_data bootstrapping and key pair support.
terraform/modules/s3/main.tf	Creates frontend S3 bucket and public access block resource.
terraform/billing_alarm.tf	Creates SNS topic and CloudWatch EstimatedCharges alarm at default USD 1 threshold.

11.1 Cloud economics and cleanup

The billing alarm and stop script show cloud economics awareness. The default billing threshold is USD 1 per month in variables.tf, and the captured teardown destroyed 14 resources. This is important for coursework because cloud demos can otherwise create unintended recurring costs.

11.2 IaC accuracy notes

- The current Terraform has one public subnet rather than two public subnets.
- The deployment is useful for demonstration but is not multi-AZ high availability.
- The S3 module disables public access block, which should be revisited for production.
- The EC2 module uses user_data scripts, which gives zero-touch provisioning but can become harder to maintain as complexity grows.

12. Docker and Kubernetes Orchestration

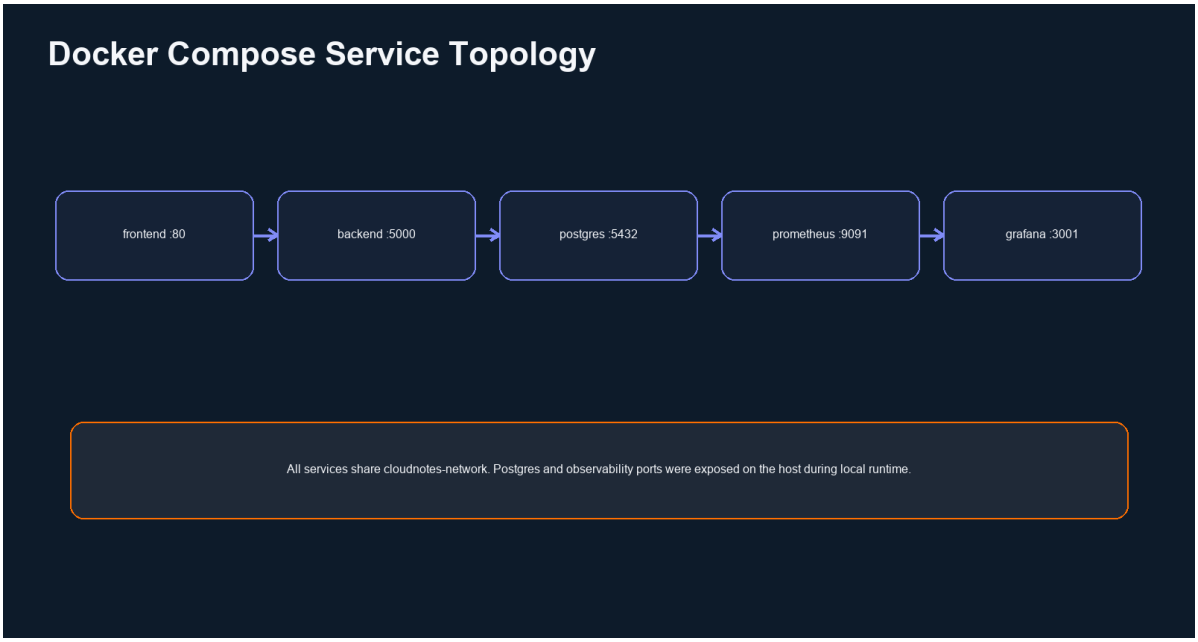


Figure 6. Docker Compose service topology.

12.1 Docker Compose services

Service	Image/build	Ports	Role
db	postgres:15-alpine	5432:5432	Stores CloudNotes note data with pgdata volume
backend	Builds ../app/backend	5000:5000	Flask API and metrics endpoint
frontend	Builds ../app/frontend	80:80	Nginx static frontend and reverse proxy
prometheus	prom/prometheus:latest	9091:9090	Scrapes backend and Prometheus metrics
grafana	grafana/grafana:latest	3001:3000	Visualizes Prometheus metrics

12.2 Kubernetes implementation

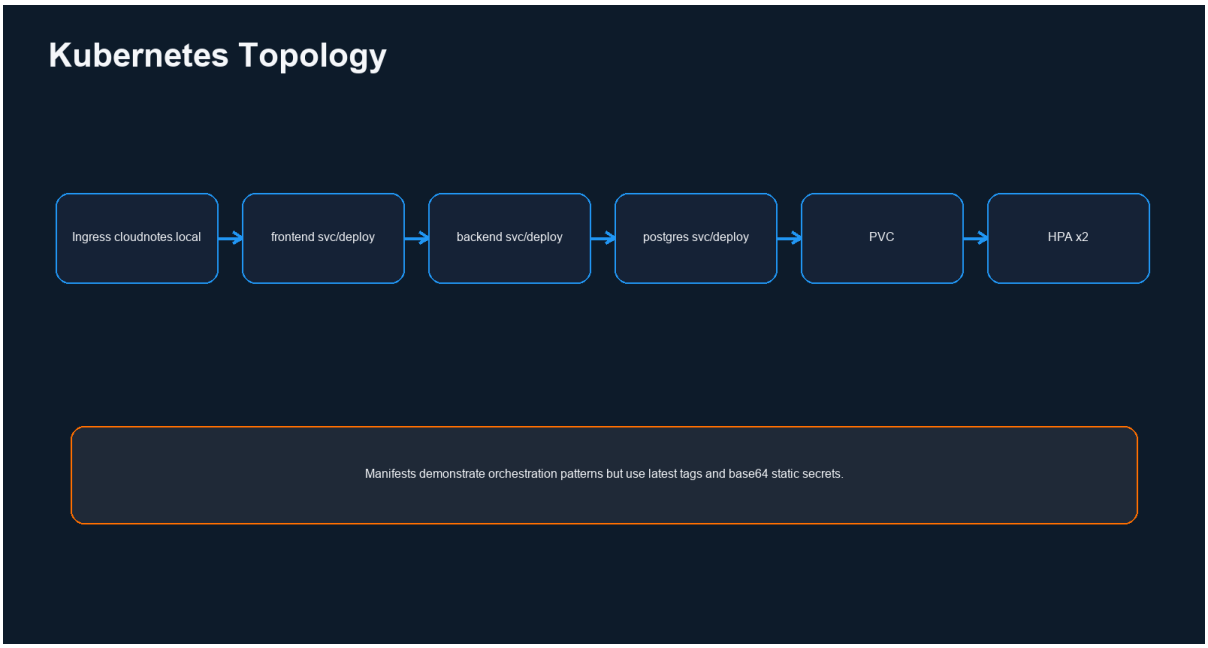


Figure 7. Kubernetes deployment topology.

Kubernetes resource	Implementation detail
Namespace	cloudnotes namespace isolates project resources.

Kubernetes resource	Implementation detail
Backend Deployment	2 replicas, image savinaysingh7/cloudnotes-backend:latest, liveness/readiness probes on /api/health.
Frontend Deployment	2 replicas, NodePort service on 30080.
Postgres Deployment	1 replica, postgres:15-alpine, PVC-mounted storage.
Secret	Opaque Secret stores database URL and PostgreSQL password as base64-encoded values.
PVC	postgres-pvc requests 1Gi storage.
HPA	Backend scales 2 to 5 at 70 percent CPU; frontend scales 2 to 4 at 80 percent CPU.
Ingress	nginx ingress routes /api to backend and / to frontend for cloudnotes.local.

13. Continuous Integration and Deployment

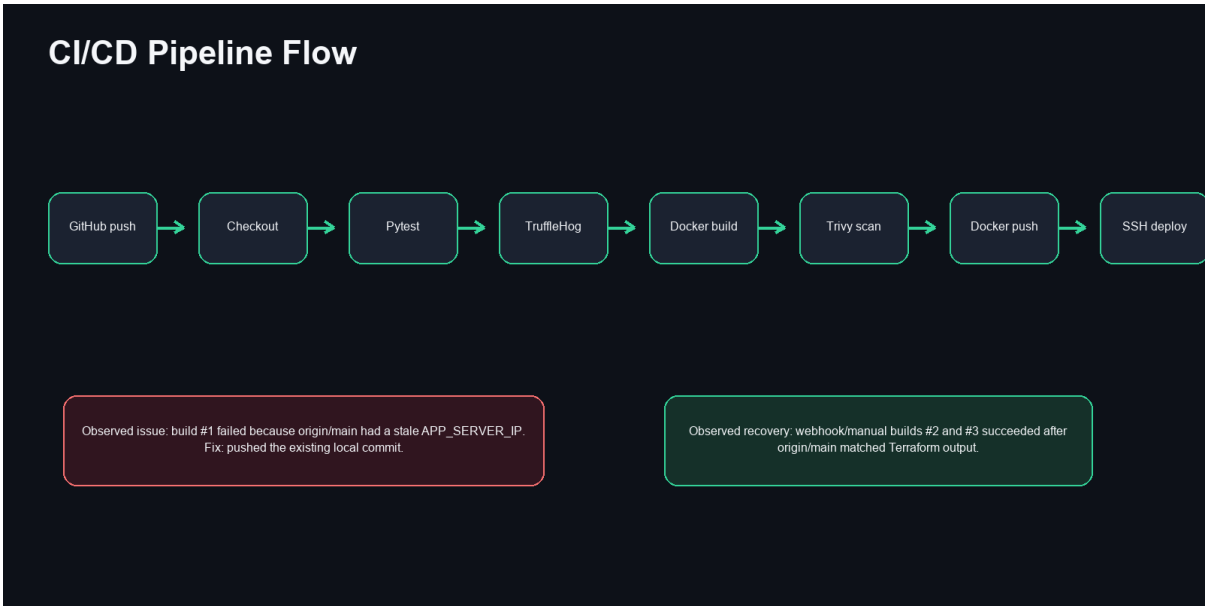


Figure 8. Jenkins CI/CD flow.

The Jenkinsfile implements pipeline as code. It disables concurrent builds, responds to GitHub push events, defines Docker Hub image names and build tags, and performs a sequence of stages from checkout to AWS deployment. The captured Jenkins evidence shows one failed build caused by stale APP_SERVER_IP in origin/main, followed by successful builds after source synchronization.

Pipeline stage	Purpose	DevOps principle
Code Checkout	Fetch source from GitHub	Version-controlled source of truth
Backend Tests	Build backend test image and run pytest	Continuous integration feedback
Security Scan	Run TruffleHog with only-verified output	Shift-left secret detection
Build Docker Images	Tag backend test image and build frontend image	Immutable packaging
Image Vulnerability Scan	Run Trivy for HIGH/CRITICAL findings	DevSecOps image scanning
Push to Docker Hub	Publish build-number and latest tags	Artifact registry integration
Deploy to AWS Production	SCP compose/env/monitoring files and run docker compose pull/up	Automated deployment

13.1 Jenkins configuration

- jenkins/Dockerfile.local builds a Jenkins LTS JDK 21 image with GitHub, workflow, Docker, credentials, JCasC, Job DSL, SSH, cleanup, Stage View, and Graph View plugins.
- jenkins/jenkins.yaml disables signup, creates an admin user, sets the Jenkins URL, and defines the CloudNotes-Pipeline job from the GitHub repository.
- jenkins/init.groovy.d/creds.groovy injects Docker Hub and app server SSH credentials from mounted secret files.

Build	Result	Duration	Started
1	FAILURE	232.4 sec	2026-05-26 14:58:28 +05:30
2	SUCCESS	64.1 sec	2026-05-26 15:03:17 +05:30
3	SUCCESS	48.6 sec	2026-05-26 15:04:30 +05:30

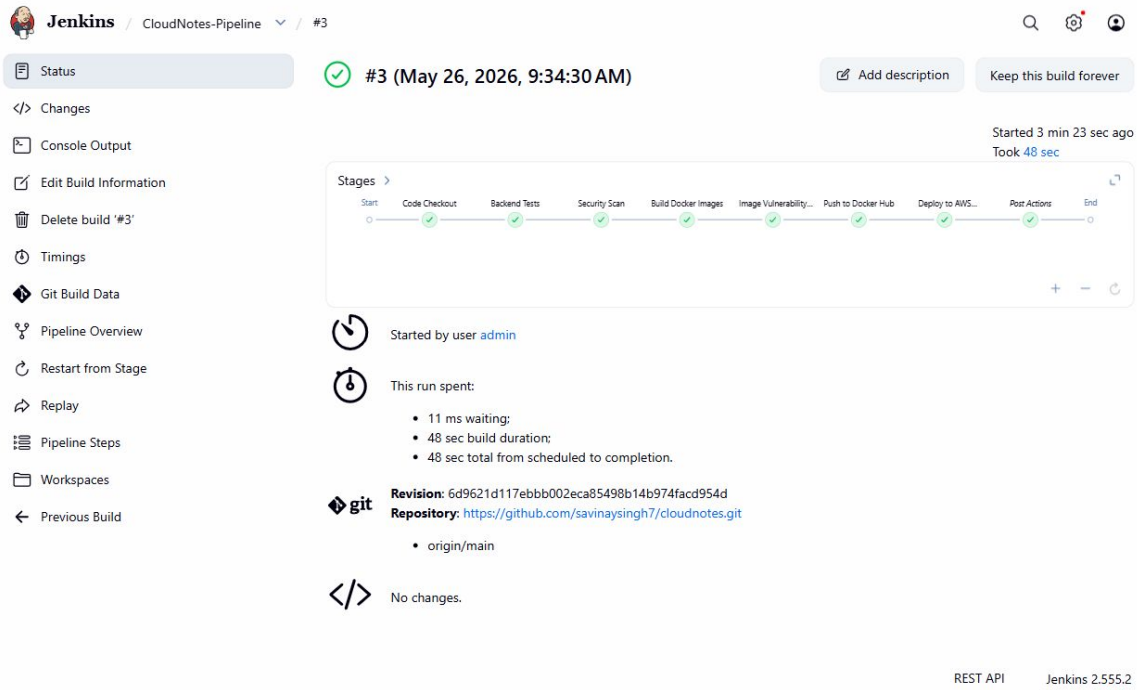


Figure 9. Jenkins successful build evidence.

14. Monitoring, Logging and Observability

14.1 Prometheus

Prometheus is configured with a 15-second scrape interval and evaluation interval. It scrapes Prometheus self-metrics and the CloudNotes backend at backend:5000/metrics. The backend exports default HTTP metrics and custom counters for notes_created_total, notes_deleted_total, and notes_updated_total.

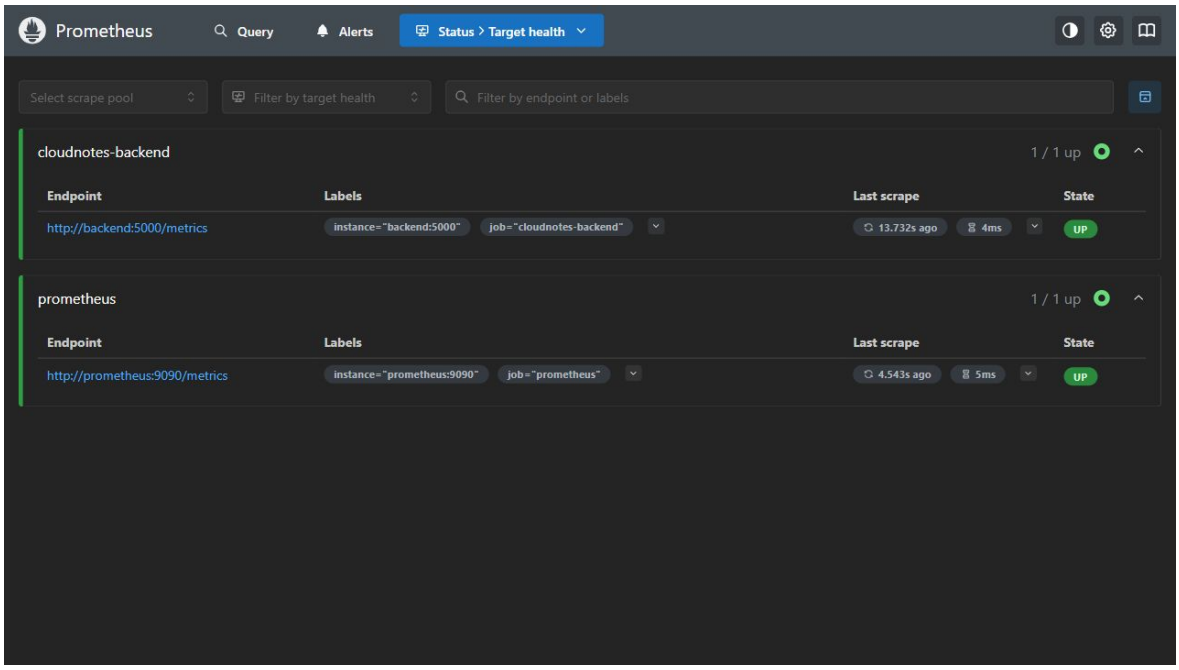


Figure 10. Prometheus targets page.

14.2 Alert rules

Alert	Condition	Severity
BackendDown	up{job="cloudnotes-backend"} == 0 for 1 minute	critical
HighErrorRate	5xx request ratio greater than 10 percent for 2 minutes	critical
HighRequestLatency	P95 latency greater than 1 second for 3 minutes	warning
NoNotesCreated	No new notes in 1 hour after prior activity	info

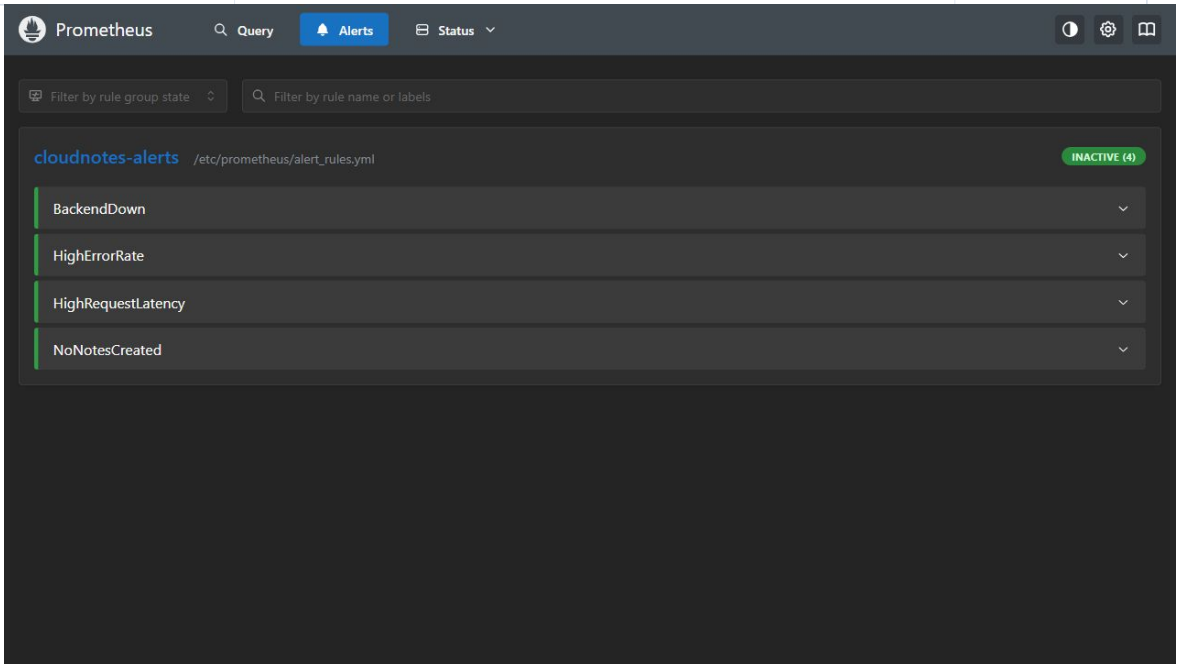


Figure 11. Prometheus alert rules evidence.

14.3 Grafana dashboard

The dashboard file monitoring/grafana/provisioning/dashboards/cloudnotes.json currently defines eight panels, expanding beyond the six-panel baseline in the assignment prompt. The panels are Backend Status, Total Notes Created, Total Notes Deleted, Total Notes Updated, Error Rate, HTTP Request Rate, P95 Latency, and Backend Memory Usage.

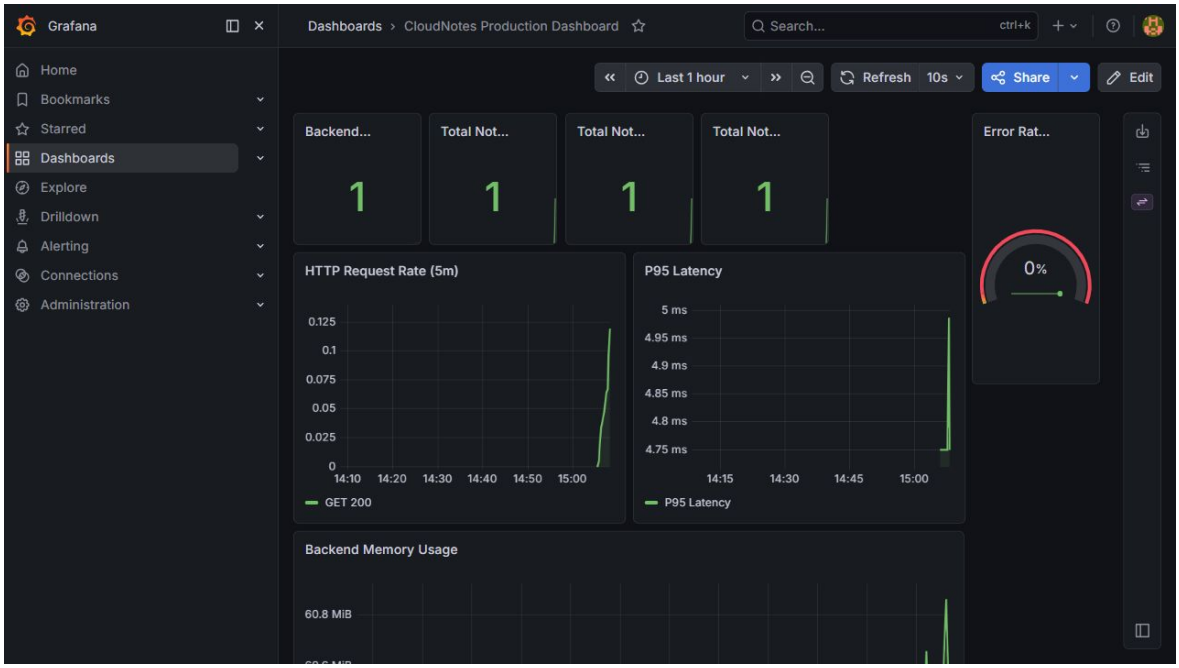


Figure 12. Grafana CloudNotes Production Dashboard.

15. Security and Best Practices

CloudNotes demonstrates DevSecOps awareness by integrating security checks into CI/CD and by using Kubernetes Secrets and Jenkins credential injection. At the same time, the project is a learning demo, so the report must distinguish implemented practices from production hardening that should be added before real enterprise use.

Security area	Implemented practice	Production recommendation
Secret scanning	TruffleHog runs in Jenkins and fails on verified findings.	Also scan pull requests and rotate any historical secrets.
Image scanning	Trivy scans backend images for HIGH/CRITICAL vulnerabilities.	Make critical findings blocking; archive scan reports.
Jenkins credentials	Docker Hub and SSH keys are injected through Jenkins credentials.	Move to short-lived tokens and managed secret storage.
Kubernetes Secrets	Database values are separated from deployment manifests.	Use SealedSecrets or External Secrets because base64 is not encryption.
Network control	Security groups explicitly declare exposed ports.	Restrict admin/monitoring/SSH ports to VPN or known CIDR ranges.
Application safety	Frontend escapes note title/body during rendering.	Add authentication, authorization, rate limiting, and stronger validation.
Cloud cost safety	Billing alarm and stop script are included.	Use budgets, tags, and automated cleanup policies.

15.1 Security considerations observed

- The API has no user authentication, so a real deployment would need identity and authorization before internet exposure.
- Several demo credentials appear in configuration files and should be treated as disposable for coursework only.
- The AWS security groups are broad for demonstration convenience. Production deployments should not expose SSH, Jenkins, Grafana, Prometheus, and backend ports to the public internet.
- The backend Dockerfile packages the runtime successfully, but it does not currently use a non-root user or a multi-stage build.
- The Terraform EC2 root volumes and metadata options should be hardened for production by enabling encryption and requiring IMDSv2.

16. Testing and Verification

16.1 Backend unit tests

Backend tests live in app/backend/tests/test_app.py and cover health, note creation, missing-field validation, listing, search, update, delete, and 404 behavior. Running pytest from app/backend is the correct context because tests import from app.py directly.

```
Command: cd app/backend; ..\..\analysis_artifacts\backend_venv\Scripts\python.exe -m pytest
tests -q
Result: 9 passed, 4 SQLAlchemy LegacyAPIWarning warnings in 1.24 seconds
```

16.2 End-to-end browser automation

tests/e2e/puppeteer_test.js launches Chromium through Puppeteer, captures local and AWS app screenshots, logs into Grafana when possible, and captures Prometheus and Jenkins pages. This script supports visual evidence collection for demonstrations and reports.

16.3 Infrastructure verification

Verification item	Observed result
Terraform validate	Success: configuration is valid.
Captured Terraform plan	Before teardown, infrastructure matched configuration with no changes.
Captured startup	start_mumbai.bat applied Terraform, installed Jenkins credentials, updated webhook, and printed service URLs.
Captured teardown	stop_mumbai.bat destroyed 14 Terraform-managed resources.
Jenkins builds	Build #1 failed due stale IP state; builds #2 and #3 succeeded after origin/main was corrected.

17. Challenges, Solutions and Lessons Learned

Challenge	Solution implemented	Lesson learned
Database persistence in containers	Docker named volume and Kubernetes PVC	Stateful services require explicit volume management.
Service startup order	Docker Compose db healthcheck and Kubernetes probes	Readiness must be modeled instead of assuming startup order.
Jenkins IP changes after Terraform runs	PowerShell updates webhook and Jenkinsfile APP_SERVER_IP	Dynamic infrastructure state should be passed at runtime or stored in SSM instead of committed to source.
Credential handling	Jenkins credential injection via Groovy and mounted secret files	Secrets should not be hardcoded; production should use managed secret stores.
Monitoring useful metrics	Custom CRUD counters plus HTTP metrics and dashboard panels	Observability is strongest when technical and business metrics are combined.
AWS cost control	stop_mumbai.bat and billing alarm	Cleanup automation is part of responsible cloud usage.

17.1 Future improvements

- Add HTTPS/TLS through an Application Load Balancer and ACM or a hardened reverse proxy.
- Move PostgreSQL to managed RDS with backups and restore testing.
- Replace dynamic IP commits with SSM Parameter Store, Terraform remote state outputs, or Jenkins parameters.
- Add user authentication, authorization, and note ownership.
- Use Alembic/Flask-Migrate for database schema changes.
- Implement blue-green or canary deployments.
- Add centralized logging and tracing with a stack such as Loki/ELK plus OpenTelemetry.
- Harden containers with non-root users, security contexts, read-only filesystems where possible, and immutable image tags.

18. Course Outcomes Assessment

Course outcome	Assessment	Evidence
CO1: Cloud fundamentals	Demonstrated through AWS EC2/VPC/S3/SNS/CloudWatch and live deployment documentation.	terraform/*.tf, HOW_TO_RUN.md, start_mumbai.log
CO2: Virtualization and containerization	Demonstrated through Docker images, Docker Compose, and Kubernetes manifests.	app/*/Dockerfile, docker/docker-compose.yml, kubernetes/*.yaml
CO3: Infrastructure as Code	Demonstrated through modular Terraform with reproducible apply/destroy workflow.	terraform/modules/*, start_automated_terraform.ps1
CO4: CI/CD pipelines	Demonstrated through Jenkins pipeline as code and captured successful build evidence.	Jenkinsfile, jenkins_build_summary.json, Jenkins screenshot
CO5: Monitoring and security	Demonstrated through Prometheus, Grafana, alerts, TruffleHog, Trivy, and credential injection.	monitoring/*, Jenkinsfile, jenkins/init.groovy.d/creds.groovy
CO6: Advanced operations	Partially demonstrated through HPA, cost alarm, teardown automation, latency panels, and production-hardening recommendations.	kubernetes/hpa.yaml, billing_alarm.tf, cloudnotes.json, stop_mumbai.bat

Overall, CloudNotes is a strong INT377 project because it documents and implements the full lifecycle instead of stopping at application code. The project is suitable for academic demonstration, and the report identifies where production hardening would be required.

19. Visual Documentation

This section lists the visual evidence included or described for evaluation. Screenshots were reused from the captured analysis artifacts when available. Where a live AWS Console screenshot was not available, Terraform and AWS CLI evidence plus topology diagrams are used instead.

Visual requirement	Included evidence
CloudNotes Application UI	Figures 2, 3, and 4 show initial, create, and edit states.
Jenkins Dashboard	Figure 9 shows the successful Jenkins build page.
Grafana Dashboard	Figure 12 shows the CloudNotes Production Dashboard.
Prometheus Interface	Figures 10 and 11 show targets and alerts.
AWS Console / Infrastructure	Figure 5 shows AWS topology; start/stop logs show Terraform-created resources and teardown.
Metrics endpoint	Prometheus and Grafana figures show metrics consumption; /metrics is defined in app/backend/app.py.
Kubernetes Pods	Figure 7 documents Kubernetes topology; kubectl command sequence is provided in the deployment guide.
Docker Containers	Figure 6 documents Docker Compose topology.
BAT files in action	Section 10 includes BAT content and start_mumbai.log excerpt.

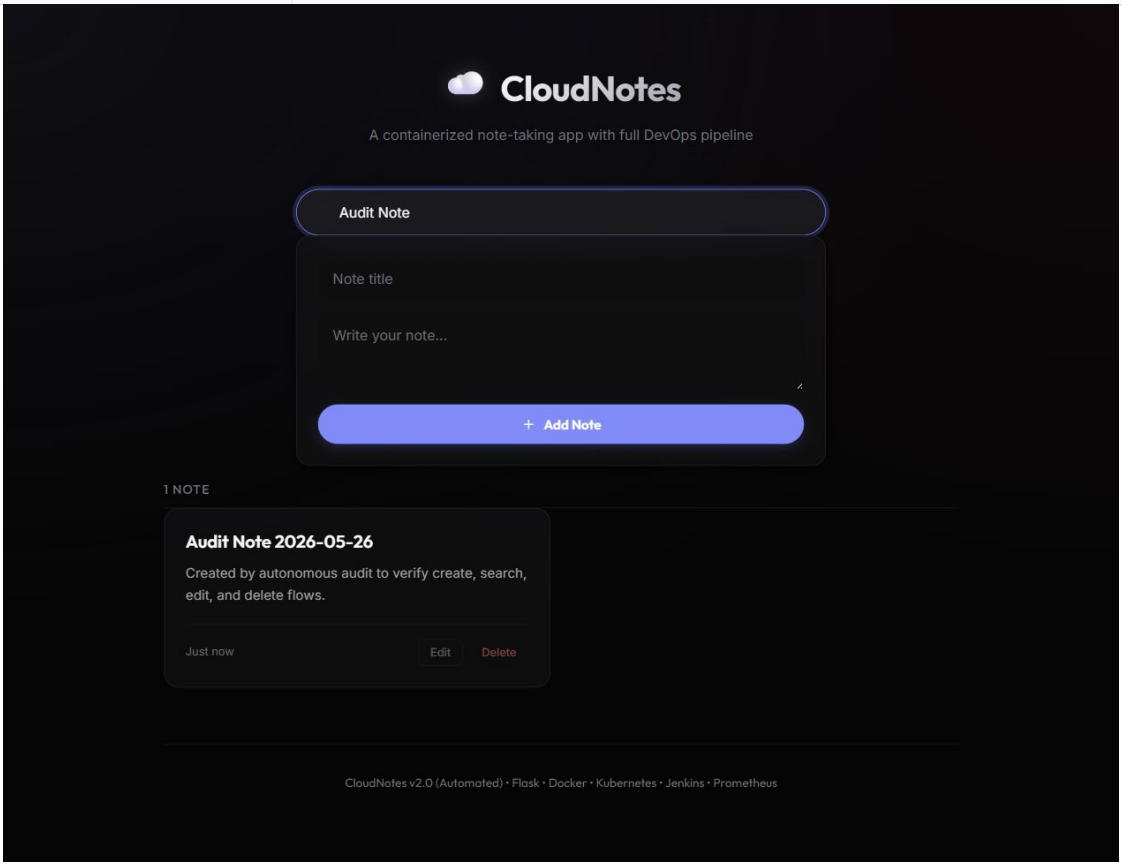


Figure 13. Search workflow evidence.

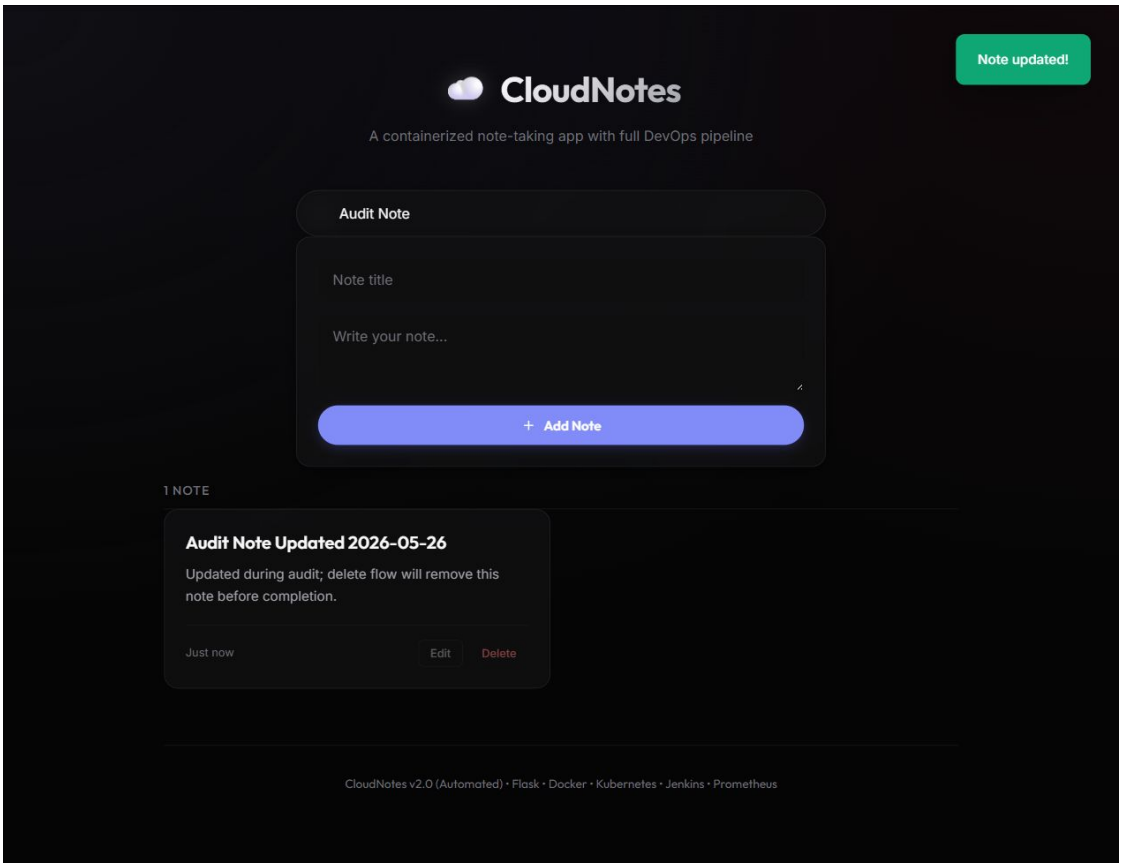


Figure 14. Updated note evidence.

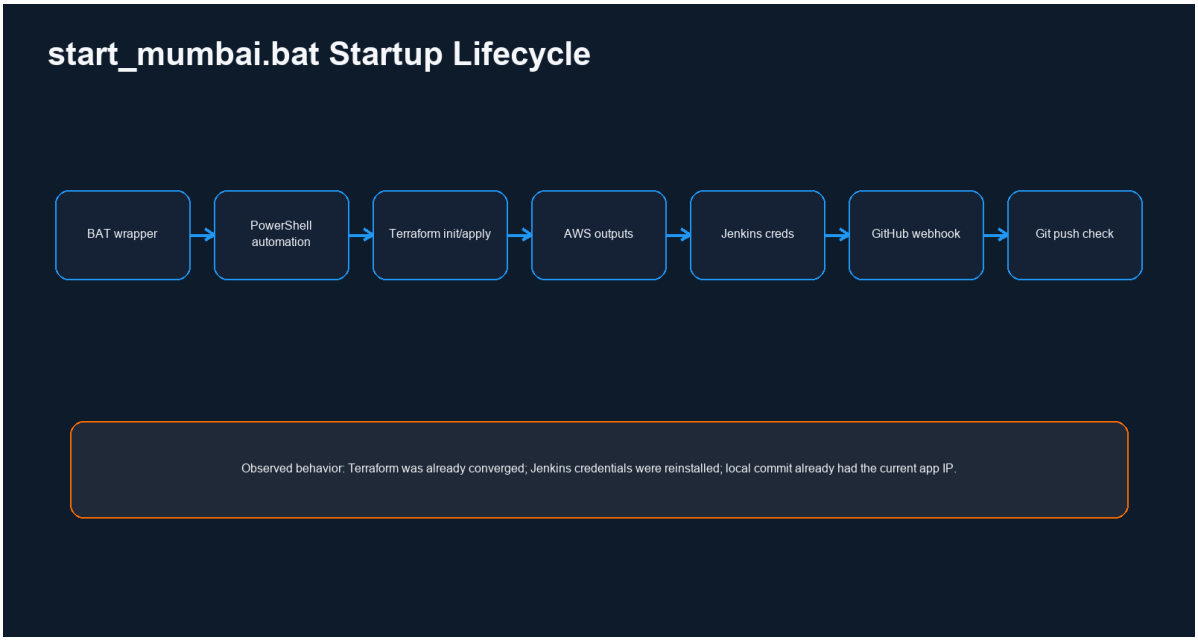


Figure 15. start_mumbai.bat automation flow.

20. Conclusion and Recommendations

CloudNotes successfully demonstrates a modern DevOps learning project for INT377. The project combines a usable CRUD application with Docker, Kubernetes, Terraform, AWS, Jenkins, Prometheus, Grafana, security scanning, testing, and automation scripts. The BAT files are especially important because they turn cloud provisioning and cleanup into repeatable operations, which is a core Infrastructure as Code and cloud-economics practice.

The strongest aspect of the project is integration. Each component has a clear role: Flask provides the API, PostgreSQL persists data, Nginx serves and proxies the frontend, Docker packages services, Kubernetes demonstrates orchestration, Terraform creates infrastructure, Jenkins automates delivery, Prometheus collects metrics, Grafana visualizes behavior, and scripts coordinate the workflow.

20.1 Final recommendations

- Keep the current report, README, HOW_TO_RUN.md, and VIVA.md together for evaluation because they explain both implementation and oral-defense points.
- Before any future public demo, rotate all credentials used during the project and avoid sharing private keys or access-key CSV files.
- Add authentication and HTTPS before treating the app as production-ready.
- Use immutable Docker tags in production instead of latest.
- Move dynamic deployment values out of Jenkinsfile commits and into runtime configuration.
- Continue using stop_mumbai.bat immediately after demonstrations to prevent unnecessary AWS charges.

With these improvements, CloudNotes can evolve from a strong academic DevOps demonstration into a more production-aligned cloud-native application.

Appendix A. Key File Inventory

Area	Files inspected
Architecture and setup	HOW_TO_RUN.md, README.md, docker/docker-compose.yml, start_automated_terraform.ps1, start_mumbai.bat, stop_mumbai.bat
Terraform	terraform/main.tf, provider.tf, variables.tf, billing_alarm.tf, modules/vpc/main.tf, modules/ec2/main.tf, modules/s3/main.tf
Application	app/backend/app.py, requirements.txt, Dockerfile, tests/test_app.py, app/frontend/index.html, app.js, style.css, Dockerfile, nginx.conf
Kubernetes	namespace, backend, frontend, postgres deployment, secret, PVC, HPA, ingress manifests
CI/CD	Jenkinsfile, jenkins/Dockerfile.local, jenkins/jenkins.yaml, jenkins/init.groovy.d/creds.groovy
Monitoring	monitoring/prometheus/prometheus.yml, alert_rules.yml, Grafana datasource and dashboard JSON
Testing	tests/e2e/puppeteer_test.js and package.json

Appendix B. Selected source excerpts

```
groups:
  - name: cloudnotes-alerts
    rules:
      - alert: BackendDown
        expr: up{job="cloudnotes-backend"} == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "CloudNotes backend is DOWN"
          description: "The backend target has been unreachable for more than 1 minute."

... excerpt truncated for report clarity ...
```

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
  namespace: cloudnotes
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
... excerpt truncated for report clarity ...
```

```
pipeline {
    agent any

    options {
        skipDefaultCheckout(true)
        disableConcurrentBuilds()
    }

    triggers {
        githubPush()
... excerpt truncated for report clarity ...
```